

discussed in §9.7.

A typical problem gives N functional relations to be zeroed, involving variables $x_i, i = 0, 1, \dots, N - 1$:

$$F_i(x_0, x_1, \dots, x_{N-1}) = 0 \quad i = 0, 1, \dots, N - 1. \quad (9.6.2)$$

We let $\mathbf{x}$ denote the entire vector of values x_i and $\mathbf{F}$ denote the entire vector of functions F_i . In the neighborhood of $\mathbf{x}$, each of the functions F_i can be expanded in Taylor series:

$$F_i(\mathbf{x} + \delta\mathbf{x}) = F_i(\mathbf{x}) + \sum_{j=0}^{N-1} \frac{\partial F_i}{\partial x_j} \delta x_j + O(\delta\mathbf{x}^2). \quad (9.6.3)$$

The matrix of partial derivatives appearing in equation (9.6.3) is the *Jacobian* matrix $\mathbf{J}$:

$$J_{ij} \equiv \frac{\partial F_i}{\partial x_j}. \quad (9.6.4)$$

In matrix notation equation (9.6.3) is

$$\mathbf{F}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{F}(\mathbf{x}) + \mathbf{J} \cdot \delta\mathbf{x} + O(\delta\mathbf{x}^2). \quad (9.6.5)$$

By neglecting terms of order $\delta\mathbf{x}^2$ and higher and by setting $\mathbf{F}(\mathbf{x} + \delta\mathbf{x}) = 0$, we obtain a set of linear equations for the corrections $\delta\mathbf{x}$ that move each function closer to zero simultaneously, namely

$$\mathbf{J} \cdot \delta\mathbf{x} = -\mathbf{F}. \quad (9.6.6)$$

Matrix equation (9.6.6) can be solved by LU decomposition as described in §2.3. The corrections are then added to the solution vector,

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \delta\mathbf{x} \quad (9.6.7)$$

and the process is iterated to convergence. In general it is a good idea to check the degree to which both functions and variables have converged. Once either reaches machine accuracy, the other won't change.

The following routine `mnewt` performs `ntrial` iterations starting from an initial guess at the solution vector `x[0..n-1]`. Iteration stops if either the sum of the magnitudes of the functions F_i is less than some tolerance `tolF`, or the sum of the absolute values of the corrections to δx_i is less than some tolerance `tolx`. `mnewt` calls a user-supplied function with the fixed name `usrfun`, which must provide the function values $\mathbf{F}$ and the Jacobian matrix $\mathbf{J}$. (The more sophisticated methods later in this chapter will have a more versatile interface.) If $\mathbf{J}$ is difficult to compute analytically, you can try having `usrfun` invoke the routine `NRfdjac` of §9.7 to compute the partial derivatives by finite differences. You should not make `ntrial` too big; rather, inspect to see what is happening before continuing for some further iterations.

```

mnewt.h void usrfun(VecDoub_I &x, VecDoub_O &fvec, MatDoub_O &fjac);

void mnewt(const Int ntrial, VecDoub_IO &x, const Doub tolx, const Doub tolf) {
    Given an initial guess x[0..n-1] for a root in n dimensions, take ntrial Newton-Raphson steps
    to improve the root. Stop if the root converges in either summed absolute variable increments
    tolx or summed absolute function values tolf.
    Int i,n=x.size();
    VecDoub p(n),fvec(n);
    MatDoub fjac(n,n);
    for (Int k=0;k<ntrial;k++) {
        usrfun(x,fvec,fjac);           User function supplies function values at x in
        Doub errf=0.0;                 fvec and Jacobian matrix in fjac.
        for (i=0;i<n;i++) errf += abs(fvec[i]);    Check function convergence.
        if (errf <= tolf) return;
        for (i=0;i<n;i++) p[i] = -fvec[i];        Right-hand side of linear equations.
        LUdcmp alu(fjac);                     Solve linear equations using LU decomposition.
        alu.solve(p,p);
        Doub errx=0.0;                       Check root convergence.
        for (i=0;i<n;i++) {                   Update solution.
            errx += abs(p[i]);
            x[i] += p[i];
        }
        if (errx <= tolx) return;
    }
    return;
}

```

9.6.1 Newton's Method versus Minimization

In the next chapter, we will find that there *are* efficient general techniques for finding a minimum of a function of many variables. Why is that task (relatively) easy, while multidimensional root finding is often quite hard? Isn't minimization equivalent to finding a zero of an N -dimensional gradient vector, which is not so different from zeroing an N -dimensional function? No! The components of a gradient vector are not independent, arbitrary functions. Rather, they obey so-called integrability conditions that are highly restrictive. Put crudely, you can always find a minimum by sliding downhill on a single surface. The test of "downhillness" is thus one-dimensional. There is no analogous conceptual procedure for finding a multidimensional root, where "downhill" must mean simultaneously downhill in N separate function spaces, thus allowing a multitude of trade-offs as to how much progress in one dimension is worth compared with progress in another.

It might occur to you to carry out multidimensional root finding by collapsing all these dimensions into one: Add up the sums of squares of the individual functions F_i to get a master function F that (i) is positive-definite and (ii) has a global minimum of zero exactly at all solutions of the original set of nonlinear equations. Unfortunately, as you will see in the next chapter, the efficient algorithms for finding minima come to rest on global and local minima indiscriminately. You will often find, to your great dissatisfaction, that your function F has a great number of local minima. In Figure 9.6.1, for example, there is likely to be a local minimum wherever the zero contours of f and g make a close approach to each other. The point labeled M is such a point, and one sees that there are no nearby roots.

However, we will now see that sophisticated strategies for multidimensional root finding can in fact make use of the idea of minimizing a master function F , by *combining* it with Newton's method applied to the full set of functions F_i . While such methods can still occasionally fail by coming to rest on a local minimum of F ,

they often succeed where a direct attack via Newton's method alone fails. The next section deals with these methods.

CITED REFERENCES AND FURTHER READING:

- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 14.[1]
 Ostrowski, A.M. 1966, *Solutions of Equations and Systems of Equations*, 2nd ed. (New York: Academic Press).
 Ortega, J., and Rheinboldt, W. 1970, *Iterative Solution of Nonlinear Equations in Several Variables* (New York: Academic Press); reprinted 2000 (Philadelphia: S.I.A.M.).

9.7 Globally Convergent Methods for Nonlinear Systems of Equations

We have seen that Newton's method for solving nonlinear equations has an unfortunate tendency to wander off into the wild blue yonder if the initial guess is not sufficiently close to the root. A *global* method [1] would be one that converges to a solution from almost any starting point. Such global methods do exist for minimization problems; an example is the quasi-Newton method that we will describe in §10.9. In this section we will develop an algorithm that is an analogous quasi-Newton method for multidimensional root finding. Alas, while it is better behaved than Newton's method, it is still not truly global.

What the method *does* do is combine the rapid local convergence of Newton's method with a higher-level strategy that guarantees at least *some* progress at each step — either toward an actual root (usually), or else, hopefully rarely, toward the situation labeled “no root here!” in Figure 9.6.1. In the latter case, the method recognizes the problem and signals failure. By contrast, Newton's method can bounce around forever, and you are never sure whether or not to quit.

Recall our discussion of §9.6: The Newton step for the set of equations

$$\mathbf{F}(\mathbf{x}) = 0 \quad (9.7.1)$$

is

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \delta\mathbf{x} \quad (9.7.2)$$

where

$$\delta\mathbf{x} = -\mathbf{J}^{-1} \cdot \mathbf{F} \quad (9.7.3)$$

Here $\mathbf{J}$ is the Jacobian matrix. How do we decide whether to accept the Newton step $\delta\mathbf{x}$? A reasonable strategy is to require that the step decrease $|\mathbf{F}|^2 = \mathbf{F} \cdot \mathbf{F}$. This is the same requirement we would impose if we were trying to minimize

$$f = \frac{1}{2} \mathbf{F} \cdot \mathbf{F} \quad (9.7.4)$$

(The $\frac{1}{2}$ is for later convenience.) Every solution to (9.7.1) minimizes (9.7.4), but there may be local minima of (9.7.4) that are not solutions to (9.7.1). Thus, as already mentioned, simply applying one of our minimum-finding algorithms from Chapter 10 to (9.7.4) is *not* a good idea.